

A COOKBOOK FOR BACKEND DEVELOPERS

Data Engineering Cookbook

Move data reliably at scale — ETL/ELT, orchestration, warehouses and lakes, batch and streaming, and data quality — the pipelines every ML and analytics system runs on.

ETL / ELT

Airflow

Spark

Kafka

Warehouses

Parquet

dbt

Data quality

Written for a backend developer who moves and models data, and now needs the pipelines feeding analytics and ML.

Every chapter answers: what it solves, how to build it, the failure modes, and when a simpler approach wins.

Edition 2026 · reliable pipelines · the plumbing under the models

Table of Contents

How to Use This Book	4
The one idea: reliable pipelines from source to consumable	4
What this book covers	5
Part 1 · What Data Engineering Is	6
1.1 Where it sits	6
1.2 ETL vs ELT — the defining shift	6
1.3 The layered data architecture	7
Part 2 · Storage: Lakes, Warehouses & File Formats	8
2.1 Lake vs warehouse vs lakehouse	8
2.2 File formats: the cheap, high-impact choice	8
2.3 Partitioning	9
Part 3 · Building ETL/ELT	10
3.1 Extract: full vs incremental	10
3.2 Load: land raw, then transform (ELT)	10
3.3 Idempotency & incremental writes	10
3.4 Backfills	11
Part 4 · Orchestration	12
4.1 The DAG	12
4.2 What an orchestrator gives you	12
4.3 The tools	12
Part 5 · Batch Processing at Scale (Spark)	14
5.1 When you actually need Spark	14
5.2 The Spark model	14
5.3 The distributed gotchas	15
Part 6 · Streaming (Kafka & Real-Time Data)	16
6.1 Batch vs streaming	16
6.2 The log: Kafka	16
6.3 Stream processing	17
6.4 Exactly-once is hard	17
Part 7 · Data Modeling & Transformation (dbt)	18
7.1 Model the gold layer for consumers	18
7.2 dbt: transformations as versioned SQL	18
7.3 Staging then marts	19
Part 8 · Data Quality	20
8.1 What "bad data" looks like	20
8.2 Test data like you test code	20
8.3 Fail fast, quarantine, alert	21
8.4 Freshness and SLAs	21
Part 9 · Pipelines in Production	22
9.1 The non-negotiables (recap, because they matter)	22

9.2 Monitor the pipeline and the data	22
9.3 Handle sources that misbehave	22
9.4 Cost	23
Part 10 · Capstone & Cheat Sheet	24
10.1 Capstone — build a reliable ELT pipeline	24
10.2 Decision cheat sheet	24
10.3 Concept cheat sheet	24
10.4 The five rules.	25

How to Use This Book

Every dashboard, every analysis, every trained model sits on top of data that someone had to **move, clean, and shape reliably** — from source systems into a place it can be used, on a schedule, without breaking. That work is **data engineering**, and it's the unglamorous foundation the whole data/ML stack depends on. Your Data Analysis book taught you to *use* data; this book teaches you to *deliver* it.

It's squarely engineering, which suits you: pipelines are software — they have inputs, outputs, tests, failures, retries, and SLAs. The new ideas are about scale, scheduling, and the peculiar ways data breaks.

Each chapter answers four questions:

1. **What problem does this solve?**
2. **How do you build it (Python / SQL / tooling)?**
3. **What are the failure modes?**
4. **When is a simpler approach the right call?**

The one idea: reliable pipelines from source to consumable

A data pipeline moves data from where it's produced to where it's used, reshaping it on the way, on a schedule, reliably:

DIAGRAM

```

SOURCES → INGEST → STORE (lake/warehouse) → TRANSFORM → CONSUMERS
(apps, APIs, (batch or (raw → cleaned → (SQL/Spark) (BI, ML,
DBs, events) streaming) modeled tables) analysts)
  
```

The job is to make that flow **dependable**: correct data, on time, every time, with a clear trail when something goes wrong. Most of data engineering is achieving *reliability* over that pipeline, not the transformations themselves.

ENGINEER'S MENTAL MODEL

A pipeline is a distributed, scheduled batch job (or an event stream) with all the concerns you know — idempotency, retries, backfills, monitoring, schemas — applied to data instead of requests. If you've built a robust job queue (your Queues book), you already have the instincts; data engineering adds large-scale storage, orchestration of many interdependent jobs, and data-shaped failure modes.

What this book covers

What data engineering is and the ETL/ELT shift; **storage** (lakes, warehouses, lakehouse; file formats); **building ETL/ELT**; **orchestration** (Airflow/Dagster DAGs); **batch at scale** (Spark); **streaming** (Kafka); **data modeling** (dbt, dimensional modeling); **data quality**; and **running pipelines in production** (idempotency, backfills, SLAs). Tool-aware, principle-first.

HOW TO READ IT

Read Parts 1–4 in order — the model, storage, ETL/ELT, and orchestration are the core every pipeline uses. Parts 5–6 (Spark, streaming) are for scale and real-time; reach for them when you need them. Parts 7–9 (modeling, quality, production) turn pipelines from working to trustworthy. Build the exercises around a real source (an API or a database you have) landing into a warehouse.

READ THIS FIRST: MOST DATA ISN'T BIG DATA

The tools here (Spark, Kafka, Airflow) exist for **scale and complexity** that a lot of projects never reach. If your data fits in memory or a single Postgres, a Python script on a cron writing to a database beats a distributed stack every time — simpler, cheaper, easier to debug. Learn the big tools, but reach for them only when the data's size, velocity, or the number of interdependent jobs actually demands it.

Part 1 · What Data Engineering Is

Data engineering builds and operates the systems that move data from where it's created to where it's used. This chapter frames the discipline, the roles around it, and the one architectural shift — ETL to ELT — that defines how modern pipelines are built.

1.1 Where it sits

Role	Job	This book
Data engineer	build reliable pipelines & data platforms	← you, here
Data analyst	answer questions from the data	Data Analysis book
Data scientist / ML eng	model the data	ML/DL/MLOps books

Everyone downstream depends on the data engineer: analysts and models are only as good as the data delivered to them. "Garbage in" starts here.

1.2 ETL vs ELT — the defining shift

The classic pattern was **ETL**: Extract from sources, **Transform** in a separate processing step, then **Load** the cleaned result into a warehouse. Modern warehouses are so powerful and cheap that the order flipped to **ELT**: Extract, **Load raw data first**, then **Transform inside the warehouse** with SQL.

DIAGRAM

ETL: extract → TRANSFORM (outside) → load clean data
 ELT: extract → load RAW → TRANSFORM (in the warehouse, in SQL) ← modern default

ELT wins for most analytics/ML today because: raw data is preserved (you can re-derive transforms without re-extracting), transformations are just versioned SQL (dbt, Part 7), and the warehouse does the heavy lifting at scale.

ENGINEER'S MENTAL MODEL

ELT is "load first, transform later, in the database" — the same instinct as your Data Analysis book's "push work down to the warehouse." You keep an immutable raw layer (like keeping source data read-only) and build cleaned, modeled tables *on top* as reproducible SQL, rather than throwing away the original by transforming before load. Raw is your source of truth.

1.3 The layered data architecture

Most pipelines organize data into layers of increasing refinement (often called bronze/silver/gold, or raw/staging/marts):

- **Raw (bronze)** — data exactly as ingested, immutable. Your audit trail.
- **Cleaned/staging (silver)** — typed, deduped, validated.
- **Modeled/marts (gold)** — business-ready tables analysts and models query.

Each layer is rebuilt from the one before, so a bug is fixed by re-running a transform, never by hand-patching data.

COMMON MISTAKE

Transforming data before you store the raw version, then discovering a bug in the transform — with no way to re-derive correct data because the original is gone. **Always land raw, immutable data first** (ELT), and build cleaned layers on top. Raw data is cheap to store and priceless when (not if) you need to reprocess it after finding a mistake or adding a new derived field.

WHEN NOT TO BUILD THE FULL LAYERED STACK

A single small source feeding one dashboard doesn't need bronze/silver/gold and a warehouse — a script that pulls, cleans, and writes one table is enough. The layered architecture pays off with **many sources, many consumers, and evolving transforms**. Scale the architecture to the number of sources and consumers, not to what looks professional.

EXERCISE 1.1

Take a data need you have (e.g. "combine app events + a payments API for reporting"). Sketch it as ELT with raw/staging/marts layers: what's extracted, what lands raw, what the cleaned and modeled tables look like, and who consumes the gold layer. Identify one transform you'd want to change later and confirm your design lets you re-derive it from raw without re-extracting.

Part 2 · Storage: Lakes, Warehouses & File Formats

Where data lives shapes everything you can do with it. Data engineers choose among data lakes, warehouses, and the lakehouse, and — more day-to-day — the **file format**, which quietly determines your cost and speed. This chapter is the map.

2.1 Lake vs warehouse vs lakehouse

	Data lake	Data warehouse	Lakehouse
Stores	any files (raw, unstructured)	structured tables	files + table layer
Query	via engines (Spark, etc.)	SQL, fast	SQL on lake storage
Schema	on read	on write	on read/write
Examples	S3/GCS + files	Snowflake, BigQuery, Redshift	Databricks, Iceberg/Delta on S3

- **Warehouse** — structured, SQL-first, great for analytics/BI/ELT. The default for most analytics.
- **Lake** — cheap object storage for *anything* (raw logs, images, Parquet); flexible, needs an engine to query.
- **Lakehouse** — table formats (Delta, Iceberg, Hudi) that add warehouse-like reliability (ACID, schema) *on top of* lake storage. The converging modern answer.

2.2 File formats: the cheap, high-impact choice

For any sizable data, the format matters as much as the store:

Format	Kind	Use for
CSV / JSON	row, text	interchange, tiny data — <i>not</i> analytics at scale
Parquet	columnar, compressed	analytics — the default for data at rest
Avro	row, binary	streaming / record-by-record (Kafka)
Delta / Iceberg	table on Parquet	lakehouse tables (ACID, time travel)

Parquet is the workhorse. Columnar layout means a query reading 3 of 50 columns reads only those 3, and compression slashes size — often 10× smaller and far faster than CSV for analytics.

ENGINEER'S MENTAL MODEL

Choosing row vs columnar is choosing a data structure for the access pattern: row formats (CSV, Avro) are fast to write one record at a time (ingestion, streaming); columnar (Parquet) is fast to scan a few columns over millions of rows (analytics). Storing analytics data as CSV is like using a linked list where you needed a column store — it works, and it's needlessly slow and huge.

2.3 Partitioning

Large tables are split into files by a key (usually date: `dt=2026-09-01/...`) so queries and jobs read only the partitions they need. Good partitioning turns a full-table scan into reading one day's files — one of the biggest levers on cost and speed.

COMMON MISTAKE

Storing large analytics data as CSV or JSON and scanning whole files (or whole tables) for every query. You pay in money and minutes for data you didn't need to read. Convert to **Parquet and partition by a sensible key** (usually date); it's often a 10×+ improvement in cost and speed for near-zero effort — the single easiest win in data engineering.

WHEN NOT TO REACH FOR A LAKE/LAKEHOUSE

If your data fits comfortably in a warehouse (or even a Postgres), just use it — a lake adds an engine and operational complexity you may not need. Object storage + Parquet + a query engine earns its place at **large scale, mixed/ unstructured data, or when compute and storage must scale independently**. Don't build a lakehouse for gigabytes.

EXERCISE 2.1

Take a sizable CSV (or generate one, ~1M rows). Convert it to Parquet, partitioned by a date column, and compare: file size, and the time to run a query touching only a few columns and one date range against the CSV version. Measure the difference — you'll see why Parquet + partitioning is the default.

Part 3 · Building ETL/ELT

Now the actual work: extracting from sources, loading data, and transforming it. The patterns here — full vs incremental loads, idempotency, and where the transform lives — are what separate a fragile one-off script from a pipeline you can trust to run unattended every night.

3.1 Extract: full vs incremental

- **Full load** — pull everything every run. Simple, fine for small sources; wasteful and slow as data grows.
- **Incremental load** — pull only what changed since last run (by an `updated_at` timestamp, an id watermark, or change-data-capture). Essential at scale; the standard for large sources.

PYTHON

```
# incremental extract by watermark
last = read_watermark()                                # e.g. max updated_at from
last run
new_rows = query("SELECT * FROM orders WHERE updated_at > %s", last)
load(new_rows); write_watermark(max(r.updated_at for r in new_rows))
```

3.2 Load: land raw, then transform (ELT)

Load the extracted data **raw and immutable** into the raw layer first (Part 1), then transform in the warehouse. This preserves the source of truth and lets you re-derive any cleaned table later without re-hitting the source system (which may be slow, rate-limited, or unable to give you history).

3.3 Idempotency & incremental writes

A pipeline *will* be re-run — a retry, a backfill, a manual rerun after a fix. Each run must be **idempotent**: running it twice produces the same result, not duplicates. The common patterns:

- **Overwrite by partition** — a run for `dt=2026-09-01` replaces that partition wholesale (re-runnable by construction).
- **Upsert / merge** — insert-or-update on a key (`MERGE`), so re-processing a row updates rather than duplicates.
- **Idempotency key / dedupe** — dedupe on a natural key downstream.

ENGINEER'S MENTAL MODEL

This is the exact idempotency lesson from your Queues cookbook, applied to data: a step may run more than once, so design writes so a replay is a no-op or an update, never a duplicate. "Overwrite the partition" is the data-world equivalent of an idempotent handler — the safest default, because a re-run for a given day just rebuilds that day cleanly.

3.4 Backfills

When you add a new transform, fix a bug, or onboard history, you **backfill** — run the pipeline over past periods. Idempotent, partition-based writes make this safe: re-run each historical date and it rebuilds that partition. Design for backfills from day one; you'll always need them.

COMMON MISTAKE

A pipeline that appends without idempotency, so every retry or backfill **duplicates** rows — and since it fails silently (no error, just wrong counts), you discover it when a metric is inexplicably double. Make every write idempotent (overwrite-by-partition or upsert-on-key) before you schedule it, and test a double-run produces identical output. A pipeline you can't safely re-run is a pipeline you can't operate.

WHEN NOT TO DO INCREMENTAL

For a small source, a **full load** each run is simpler and less bug-prone than incremental watermarks — no state to track, no missed-row edge cases. Incremental extraction earns its complexity only when full loads become too slow or costly. Don't add watermark logic (and its bugs) to a table you could just reload in seconds.

EXERCISE 3.1

Build an ELT job for a source (an API or a DB table): extract incrementally by an `updated_at` watermark, land raw Parquet partitioned by date, and transform into a cleaned table in a warehouse/DuckDB. Make the write idempotent (overwrite-by-partition), then run it twice for the same day and confirm no duplicates. Finally, backfill three past days.

Part 4 · Orchestration

One pipeline is a script. A real data platform is *many* pipelines with dependencies — this can't run until that finishes, this runs daily, that one failed and needs a retry and an alert. **Orchestration** manages that: scheduling, dependencies, retries, and visibility across all your jobs. Airflow is the best-known tool; the concepts are universal.

4.1 The DAG

Work is modeled as a **DAG** (directed acyclic graph) — tasks with dependency edges. The orchestrator runs each task when its upstreams succeed, on a schedule, with retries and logging:

DIAGRAM

```
[extract orders] ┐
                  └─> [build staging] → [build marts] → [refresh dashboard]
[extract users]  ┐
                  └─>
run daily · retry on failure · alert on error · show the whole graph's status
```

4.2 What an orchestrator gives you

- **Scheduling** — cron-like triggers, plus data-aware triggers ("when yesterday's file lands").
- **Dependencies** — a task runs only after its upstreams succeed.
- **Retries & alerting** — automatic retries with backoff; notify on failure.
- **Backfills** — re-run a DAG over a date range (Part 3).
- **Observability** — one place to see every pipeline's status, logs, and history.

PYTHON

```
# Airflow-style DAG (sketch)
with DAG("daily_elt", schedule="0 2 * * *", catchup=True, default_args={"retries":
2}):
    extract = PythonOperator(task_id="extract", python_callable=extract_orders)
    staging = PythonOperator(task_id="staging", python_callable=build_staging)
    marts = PythonOperator(task_id="marts", python_callable=build_marts)
    extract >> staging >> marts          # dependency edges
```

4.3 The tools

- **Airflow** — the incumbent; huge ecosystem, Python DAGs, battle-tested.
- **Dagster / Prefect** — modern alternatives; nicer local dev, typed data-asset-oriented models, easier testing.

- **dbt** (Part 7) — orchestrates *SQL transformations* specifically; often run *by* Airflow/Dagster.

ENGINEER'S MENTAL MODEL

An orchestrator is cron plus a job queue plus a dependency graph plus a dashboard — the things you'd otherwise bolt together yourself to run many scheduled, interdependent jobs reliably. If you've ever chained cron jobs with "sleep and hope the first one finished," the DAG is the principled version: declare the dependency and let the orchestrator guarantee order, retries, and visibility.

COMMON MISTAKE

Chaining pipelines with cron and implicit timing — job B is scheduled 30 minutes after job A and "should" be done by then. The day A runs slow, B reads incomplete data and produces silently wrong results. Model the **dependency explicitly** (B runs when A *succeeds*, not at a guessed clock time) — that's the whole point of a DAG, and the bug cron can't prevent.

WHEN NOT TO USE A HEAVY ORCHESTRATOR

For one or two simple scheduled jobs, cron (or your scheduled-tasks tooling) is genuinely fine — Airflow is a service to run and maintain. Adopt an orchestrator when you have **many interdependent pipelines, real retry/alerting needs, and backfills** to manage. One daily script does not need a DAG engine.

EXERCISE 4.1

Model your Part 3 ELT job as a DAG in Airflow, Dagster, or Prefect (run it locally): separate extract → staging → marts tasks with explicit dependencies, a daily schedule, and 2 retries. Trigger it, make one task fail on purpose, and watch the orchestrator retry and alert. Then backfill the DAG over three past days.

Part 5 · Batch Processing at Scale (Spark)

When data outgrows one machine's memory, you need **distributed processing**, and **Apache Spark** is the standard. It splits data across a cluster and runs your transformations in parallel. This chapter is the mental model and the essentials — and an honest warning that you probably need it less often than you think.

5.1 When you actually need Spark

Reach for Spark when data is **too big for one machine's memory** (hundreds of GB to petabytes) or a job takes too long single-threaded even after Parquet + partitioning. Below that, **pandas / DuckDB / Polars on one big machine** is simpler, faster to develop, and cheaper. "Big data" starts higher than most people assume.

DIAGRAM

fits in memory?	→ pandas / Polars / DuckDB (one machine)	← most cases
too big for one machine?	→ Spark (a cluster)	← real big data

5.2 The Spark model

Spark distributes a **DataFrame** across a cluster and applies transformations in parallel. Two ideas define it:

- **Partitions** — the data is split into chunks processed in parallel across worker nodes.
- **Lazy evaluation** — transformations (`filter`, `groupBy`) build a plan; nothing runs until an **action** (`count`, `write`) triggers it, so Spark can optimize the whole plan.

PYTHON

```
from pyspark.sql import SparkSession, functions as F
spark = SparkSession.builder.getOrCreate()

df = spark.read.parquet("s3://lake/orders/")           # distributed read
result = (df.filter(F.col("amount") > 0)                # lazy: builds a plan
          .groupBy("country").agg(F.sum("amount").alias("rev")))
result.write.parquet("s3://lake/marts/rev_by_country/") # action: runs the whole plan
```

The API deliberately resembles pandas/SQL — if you know those (Data Analysis book), Spark's DataFrame ops read familiarly.

ENGINEER'S MENTAL MODEL

Spark is `map / reduce` over a cluster with a query optimizer: you describe the transformation, Spark decides how to run it in parallel across partitions. Lazy evaluation is the build-a-plan-then-execute idea (like a query planner) — which is why an errant `.collect()` that pulls all distributed data back to one machine is so dangerous: it undoes the whole point.

5.3 The distributed gotchas

- **Shuffles** — operations that move data between nodes (wide `groupBy`, `join`) are the expensive part; minimize and be aware of them.
- **Skew** — if one key has most of the data, one node does all the work; the job crawls.
- **Don't `.collect()` big data** — it pulls everything to the driver and OOMs.

COMMON MISTAKE

Using Spark for data that fits on a laptop. A Spark cluster has real overhead — startup, shuffles, cluster cost, and slower iteration — so on small/medium data it's *slower and pricier* than pandas or DuckDB, plus far more to operate. Prove you've outgrown a single big machine (with Parquet + partitioning) before reaching for a cluster. Most "we need Spark" is "we need a bigger instance and Parquet."

WHEN NOT TO USE SPARK

Below the memory limit of one machine, use DuckDB/Polars/pandas — they're faster to write, faster to run on that scale, and trivial to operate. Also skip Spark for **low-latency** needs (it's batch-oriented) — that's streaming (Part 6). Spark is for genuinely large *batch* processing, nothing smaller.

EXERCISE 5.1

Take a Data Analysis aggregation you did in pandas and re-express it in PySpark (run Spark locally). Note how similar the DataFrame API feels. Then reflect honestly on your real data size: would Spark actually beat DuckDB/Polars on one machine for it? Write down the data-size threshold at which you'd switch.

Part 6 · Streaming (Kafka & Real-Time Data)

Batch processes data in scheduled chunks; **streaming** processes each event as it arrives, for when minutes-old data isn't fresh enough. It's powerful and genuinely harder to operate than batch — so this chapter covers the model, the tools, and a firm push to be sure you need it before you build it.

6.1 Batch vs streaming

	Batch	Streaming
Processes	chunks on a schedule	each event as it arrives
Latency	minutes-hours	seconds or less
Complexity	lower	higher (always-on, ordering, state)
Use for	reports, ML training, most ELT	fraud, live metrics, real-time features

Default to batch. Streaming is worth its extra complexity only when the *business value* genuinely needs sub-minute freshness.

6.2 The log: Kafka

Most streaming centers on a **distributed, durable log** — **Apache Kafka** (or Kinesis/Pub/Sub). Producers append events to **topics**; consumers read them at their own pace; the log retains events so multiple consumers (and replays) work off the same stream.

DIAGRAM

```
producers → [ Kafka topic: an append-only, partitioned, durable log ] → consumers
              events retained; each consumer tracks its own offset; replayable
```

- **Topic** — a named stream of events (like a queue, but retained and multi-consumer).
- **Partitions** — split a topic for parallelism; ordering is guaranteed only *within* a partition.
- **Offset** — each consumer's position; lets it resume and lets you replay.

ENGINEER'S MENTAL MODEL

Kafka is a durable, replayable, multi-consumer commit log — think your Queues cookbook's broker, but events are *retained* (not deleted on consume) so many consumers read the same stream and you can rewind and reprocess. That retention + replay is the superpower over a plain job queue, and the reason it underpins event-driven architectures and real-time pipelines.

6.3 Stream processing

Consuming raw events is one thing; *transforming* streams (aggregations over time windows, joins) is stream processing — Kafka Streams, Flink, or Spark Structured Streaming. The hard parts are **windowing** (aggregate "the last 5 minutes"), **state**, and **late/out-of-order events** (the network reorders reality).

6.4 Exactly-once is hard

Guaranteeing each event is processed **exactly once** (not zero, not twice) across failures is genuinely difficult; many systems settle for **at-least-once** plus idempotent consumers (dedupe on an event id — your Queues idempotency lesson, again). Know which guarantee you have and design consumers accordingly.

COMMON MISTAKE

Building streaming because it sounds modern, when the requirement is really "updated every few minutes" — which batch does far more simply. Streaming means an always-on system with ordering, windowing, state, and delivery-semantics headaches, plus 24/7 operations. Confirm the business actually needs second-level freshness (fraud, live ops) before taking on that complexity; "near-real-time" is usually a frequent batch job.

WHEN NOT TO STREAM

If minutes-old data is fine — reporting, dashboards, ML training, most ELT — use batch (or micro-batch: run it every few minutes). Reserve streaming for genuine real-time decisions where staleness has a cost. The complexity delta between batch and streaming is large; don't pay it without a real-time requirement.

EXERCISE 6.1

Run Kafka locally (Docker). Write a producer that emits events to a topic and two consumers reading the same topic independently (observe both get all events and track separate offsets; replay by resetting an offset). Make a consumer idempotent by deduping on an event id, and reprocess to prove no double-counting. Then reflect: does your real use case need this, or would a 5-minute batch do?

Part 7 · Data Modeling & Transformation (dbt)

Landing raw data is half the job; the other half is turning it into clean, well-structured tables people can actually query correctly. That's **data modeling**, and **dbt** has become the standard way to do it — versioned SQL transformations with tests and docs. This is where the ELT "T" lives.

7.1 Model the gold layer for consumers

Analysts and models shouldn't query raw tables — they should query **business-ready, modeled tables** with clear grain and meaning. The classic approach is **dimensional modeling**: **fact** tables (events/measurements — orders, clicks) joined to **dimension** tables (the entities — customers, products). It makes queries intuitive and consistent.

DIAGRAM

```
dim_customers  ┌
                └─> fact_orders (one row per order, keys to each dimension)
dim_products   ┌
                └─>
analysts join facts to dimensions; everyone gets the same, correct numbers
```

7.2 dbt: transformations as versioned SQL

dbt lets you define each modeled table as a `SELECT` (a "model"), and handles dependencies, materialization, testing, and docs:

SQL

```
-- models/marts/fact_orders.sql
select o.order_id, o.customer_id, o.amount, o.created_at::date as order_date
from {{ ref('stg_orders') }} o          -- ref() wires dependencies into a DAG
where o.status = 'completed'
```

YAML

```
# schema tests run in the pipeline
models:
  - name: fact_orders
    columns:
      - name: order_id
        tests: [unique, not_null]
      - name: customer_id
        tests: [{ relationships: { to: ref('dim_customers'), field: customer_id } }]
```

dbt builds the dependency DAG from `ref()`, runs models in order, and runs the tests — turning your warehouse transforms into tested, documented, version- controlled software.

ENGINEER'S MENTAL MODEL

dbt makes SQL transformations behave like application code: models are functions, `ref()` declares dependencies (dbt builds the DAG), tests are unit tests on your data, and everything is in Git and code-reviewed. It's the discipline you already apply to code, finally applied to the SQL that shapes your data — no more untracked, untested transform scripts nobody understands.

7.3 Staging then marts

Standard dbt structure mirrors the layers (Part 1): **staging** models clean and standardize raw sources one-to-one; **marts** models join and aggregate staging into the dimensional, business-ready tables. Build up in reviewable steps rather than one giant query.

COMMON MISTAKE

Business logic scattered across dozens of ad-hoc SQL scripts, BI-tool queries, and notebooks — so "revenue" is computed five slightly different ways and no two reports agree. Centralize transformations in **one tested, version-controlled layer** (dbt), and have consumers read the modeled tables. One definition of each metric, reviewed and tested, is the cure for "why don't these numbers match?"

WHEN NOT TO ADD DBT / DIMENSIONAL MODELING

For a couple of simple tables feeding one dashboard, a plain SQL script (or a view) is enough — dbt and star schemas are for **many models, many consumers, and shared metric definitions**. Don't impose a modeling framework on a project too small to benefit; add it when inconsistency and duplication become real pain.

EXERCISE 7.1

With dbt (against DuckDB/Postgres/a warehouse), build staging models that clean two raw sources, then marts models forming a small star schema (one fact, two dimensions) via `ref()`. Add `unique/not_null/relationships` tests, run `dbt build`, and confirm the DAG runs in order and the tests pass. Break a test (introduce a duplicate key) and watch dbt catch it.

Part 8 · Data Quality

A pipeline that runs perfectly and delivers *wrong data* is worse than one that fails — because nobody notices, and every downstream report and model is quietly poisoned. Data quality is the discipline of catching bad data **before** it spreads. It's testing, applied to data instead of code.

8.1 What "bad data" looks like

- **Schema drift** — a source adds/removes/renames a column, or changes a type.
- **Missing or late data** — an expected file/partition never arrives, or arrives half-empty.
- **Volume anomalies** — 10× the usual rows (a duplicate load) or a tenth (a broken extract).
- **Value violations** — nulls where there shouldn't be, negatives in an amount, out-of-range dates, broken foreign keys.
- **Duplication** — the idempotency failures from Part 3.

8.2 Test data like you test code

Add automated checks at pipeline boundaries — especially right after ingestion, so bad data is caught before it flows downstream:

PYTHON

```
# assertion-style checks (or use Great Expectations / dbt tests / Soda)
assert df["order_id"].is_unique, "duplicate order_ids"
assert df["amount"].ge(0).all(), "negative amounts"
assert df["created_at"].notna().all(), "null timestamps"
assert 0.5 < len(df) / expected_rows < 2, "row count anomaly" # volume
sanity
assert set(df.columns) >= REQUIRED_COLUMNS, "schema drift: missing columns"
```

Tools: **Great Expectations**, **dbt tests** (Part 7), **Soda**, or plain assertions. The tool matters less than *having checks that fail loudly*.

ENGINEER'S MENTAL MODEL

Data-quality checks are unit tests and input validation for your pipeline — schema checks are type checks, uniqueness/not-null are assertions, volume checks are sanity bounds. And like validating an API request at the boundary, you validate incoming data at ingestion so a bad source **fails fast** at the door instead of silently corrupting every table downstream.

8.3 Fail fast, quarantine, alert

When a check fails, **stop the pipeline** (don't publish bad data), route the bad records to a quarantine/dead-letter location for inspection, and alert a human. A loud failure that halts the bad batch is a good day; silently publishing it is the incident.

8.4 Freshness and SLAs

Beyond correctness, consumers care about **timeliness**: "the sales table is updated by 6am." Monitor freshness (when did each table last update?) and alert when data is late — a stale table that looks current is its own kind of bad data.

COMMON MISTAKE

Trusting sources to stay consistent and adding no checks — then a source silently renames a column or ships nulls, the pipeline runs "successfully," and a week later someone notices every downstream number is wrong (and now the bad data is everywhere). Validate at ingestion and **fail the pipeline** on violations; a halted batch is recoverable, silently propagated bad data often isn't.

WHEN NOT TO OVER-TEST

You can't assert everything — a thousand brittle checks become noise everyone ignores, and a failing check nobody trusts is worse than none. Focus on the checks that matter: schema, key uniqueness/not-null on critical columns, volume sanity, and freshness. Test the properties whose violation would actually break a consumer, not every conceivable field.

EXERCISE 8.1

Add a validation stage to your Part 3 pipeline (assertions or Great Expectations): schema/required columns, uniqueness + not-null on the key, a volume sanity bound, and a value range. Make it **halt and alert** on failure, routing bad rows to a quarantine table. Then feed it a source with a renamed column and nulls, and confirm it fails fast instead of publishing.

Part 9 · Pipelines in Production

The difference between a pipeline that works once and one you can rely on for years is operational discipline: idempotency and backfills (recapped and cemented here), monitoring, cost control, and handling the endless small ways sources misbehave. This chapter is the operator's playbook.

9.1 The non-negotiables (recap, because they matter)

- **Idempotency** — re-running any job produces the same result, never duplicates (Part 3). This is what makes retries and backfills safe; without it, nothing else is trustworthy.
- **Backfills** — you *will* reprocess history (new transform, bug fix, new source); design partition-based, re-runnable jobs so it's routine.
- **Data quality gates** — validate at ingestion, fail fast (Part 8).

9.2 Monitor the pipeline and the data

Two layers, like MLOps:

- **Pipeline health** — did each job run, succeed, on time? Failures, durations, retries. Your orchestrator (Part 4) surfaces this; alert on failure and on jobs running long.
- **Data health** — freshness, volume, and quality metrics over time (Part 8), not just "did the job exit 0." A job can succeed and still deliver bad data.

9.3 Handle sources that misbehave

Upstream systems are outside your control and will: change schemas without warning, go down, deliver late, backfill their *own* history (restating past data), and send duplicates. Defensive design:

- **Schema-on-read tolerance** or explicit schema checks that fail clearly.
- **Idempotent, restatement-friendly** loads (a source re-sending last month shouldn't double it).
- **Retries with backoff** for flaky sources; **alerts** for missing data.

ENGINEER'S MENTAL MODEL

Treat every upstream source as an unreliable external API (your HTTP-clients and Queues instincts): it will fail, change, and lie about its data, so you validate inputs, retry transient failures, make writes idempotent, and alert when it goes quiet. A data pipeline is a long-running integration with systems you don't control — defensiveness at the boundary is the whole game.

9.4 Cost

Cloud data stacks can get expensive fast — full-table scans, un-partitioned data, oversized warehouses, always-on clusters. The big levers: **Parquet + partitioning + column pruning** (Part 2), incremental over full loads (Part 3), right-sized warehouse compute, and turning off idle clusters. Watch spend like you watch latency.

COMMON MISTAKE

"It ran green, so it's fine." A pipeline can succeed every day while silently delivering stale, incomplete, or duplicated data because the *job* worked but the *data* didn't. Monitor **data** health (freshness, volume, quality), not just job exit codes — the most damaging data incidents are the ones with a green pipeline and quietly wrong tables.

WHEN NOT TO BUILD BESPOKE INGESTION

For common sources (Salesforce, Stripe, Postgres, Google Sheets), a managed connector (Fivetran, Airbyte) often beats hand-writing and maintaining extraction — you're not paid to maintain an API client for the hundredth SaaS. Build custom ingestion for **proprietary or unusual sources**; buy/adopt connectors for the standard ones, and spend your effort on the transforms and quality that are actually yours.

EXERCISE 9.1

Make your pipeline production-grade: confirm idempotency (double-run test), add freshness + volume monitoring with alerts, and wire failure alerts from the orchestrator. Then simulate three misbehaving-source scenarios — a renamed column, a late/missing partition, and a duplicate re-send — and confirm the pipeline fails loudly or absorbs them idempotently rather than corrupting downstream tables.

Part 10 · Capstone & Cheat Sheet

10.1 Capstone — build a reliable ELT pipeline

Take one real source to a modeled, monitored, scheduled table:

1. **Design** it as ELT with raw/staging/marts layers (Part 1).
2. **Extract** incrementally by watermark; **land raw** Parquet, partitioned by date (Parts 2-3).
3. **Idempotent load** — overwrite-by-partition or upsert; prove a double-run doesn't duplicate (Part 3).
4. **Orchestrate** — an Airflow/Dagster DAG: extract → staging → marts, scheduled, with retries (Part 4).
5. **Model** — dbt staging + a small star schema (fact + dimensions) with tests (Part 7).
6. **Validate** — quality gates at ingestion that fail fast and quarantine bad rows (Part 8).
7. **Operate** — freshness/volume monitoring, failure alerts, and a clean backfill over history (Part 9).

Right-size it: reach for Spark (Part 5) or streaming (Part 6) only if data size or real-time need actually demands them — most capstones shouldn't.

10.2 Decision cheat sheet

DIAGRAM

Data size?	
fits in memory / one machine	→ pandas / DuckDB / Polars (not Spark)
too big for one machine (batch)	→ Spark
Freshness need?	
minutes-hours is fine	→ batch (or micro-batch) ← usually
seconds / real-time decisions	→ streaming (Kafka + stream processing)
Transform where?	
modern analytics/ML	→ ELT: load raw, transform in-warehouse (dbt)
Ingestion?	
standard SaaS/DB source	→ managed connector (Fivetran/Airbyte)
proprietary/unusual source	→ custom extract
Scheduling?	
one/two simple jobs	→ cron
many interdependent pipelines	→ orchestrator (Airflow/Dagster/Prefect)

10.3 Concept cheat sheet

DIAGRAM

ETL vs ELT	transform-then-load vs load-raw-then-transform (modern)
raw/staging/marts	bronze/silver/gold layers of increasing refinement
Parquet	columnar, compressed — the default for analytics at rest
partitioning	split by key (usually date) → read only what you need
incremental load	pull only what changed (watermark/CDC) vs full reload
idempotency	re-run = same result, no duplicates (overwrite-partition/upsert)

backfill	re-run a pipeline over past periods
DAG	tasks + dependencies; the orchestrator's unit of work
fact / dimension	measurements / the entities they describe (star schema)
Kafka topic	durable, replayable, multi-consumer event log

10.4 The five rules

1. **Land raw, immutable data first (ELT).** You can always re-derive; you can't un-delete the source.
2. **Make every write idempotent.** Re-runs and backfills are routine, not emergencies.
3. **Model transformations as tested, version-controlled code** (dbt) — one definition of each metric.
4. **Validate data at the boundary and fail fast.** Silently wrong data beats a loud failure to no one.
5. **Reach for big tools only at big scale.** Most data isn't big data — pandas/ DuckDB + a warehouse beats Spark/Kafka until it genuinely can't.

YOU'RE READY

You can now design and operate reliable pipelines: raw-first ELT, idempotent incremental loads, orchestrated DAGs, dbt-modeled tables, quality gates, and production monitoring — the plumbing every analytics and ML system runs on. And you know the most valuable move in data engineering: reaching for the *simplest* thing that reliably delivers the data, and scaling up only when the data forces it. Now go feed the models something clean.